Reinforcement Learning

Part 2: learning from experience

Part 2 of 7. The map is taken away, and something finally learns.

Last time we cheated

Every number in the previous lecture came out of the Bellman equation:

$$V^*(s) = \max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V^*(s') \right]$$

Look at the red term

To compute V^* you had to sum over *every* state you might land in, weighted by **how likely** it was. That is the rulebook of the universe, and you were handed it.

A robot has no table of how its wheels slip on this carpet. A chatbot has no model of how a human will react to a sentence. **Nobody gets P.**

So: can you still find π^* if the only thing you are allowed to do is **act, and see what happens**?

Outline

Four numbers at a time

No model, no labels. Just what happened when you tried.

The whole interface

From here to the end of the chapter, the agent's only access to the world is a function you can call:

```
s_next, r, done = env.step(a)
```

What you can see

One transition (s, a, r, s') at a time. Do it repeatedly and you get trajectories.

What you cannot see

$P(s'|s, a)$ – the *distribution*. You get one sample from it, never the shape.

That difference is the whole subject of this lecture. Everywhere the previous lecture wrote $\sum_{s'} P(s'|s, a)\,[\cdots]$ – an exact average – we now have to write "and we saw s' once". Replacing an expectation with samples is the entire trick.

Just play it out

The most honest estimator there is, and the most patient.

Monte Carlo: average what actually happened

$V^\pi(s)$ is defined as an expected return. You cannot compute the expectation, but you can **sample** it: play episodes, and average the returns you actually collected from s.

1. Run an episode to the end with π.
2. For each state visited, compute the actual return G_t that followed it.
3. $V(s) \leftarrow V(s) + \alpha\,[\,G_t - V(s)\,]$, or just keep a running average.

First-visit

Only count the *first* time s appears in an episode. Each return is then an independent sample – easier to prove things about.

Every-visit

Count every occurrence. Samples are correlated, the proof is harder, and in practice it works just as well.

What Monte Carlo buys and what it costs

Unbiased

G_t *is* a sample of the thing you want. No approximation, no assumption. Average enough of them and you converge to the truth.

It does not even need the Markov property.

High variance, and slow

G_t sums the randomness of *every* step that followed. One unlucky slip 40 steps later changes the number you learn from.

And you must wait for the episode to end. In a continuing task, it never does.

The practical objection. A chess agent plays 80 moves and loses. Monte Carlo tells all 80 positions they were bad. Most of them were fine. You have to average over an enormous number of games before that noise cancels out.

Learn from a guess

The idea Sutton called the central one in reinforcement learning.

Bootstrapping

Monte Carlo waits for G_t because it wants the true return. But the Bellman equation already told us what the return looks like:

$$G_t = r_t + \gamma G_{t+1} \qquad \text{and} \qquad V(s_{t+1}) \approx \mathbb{E}[G_{t+1}]$$

So after **one step** you already have an estimate of the whole future. Use it:

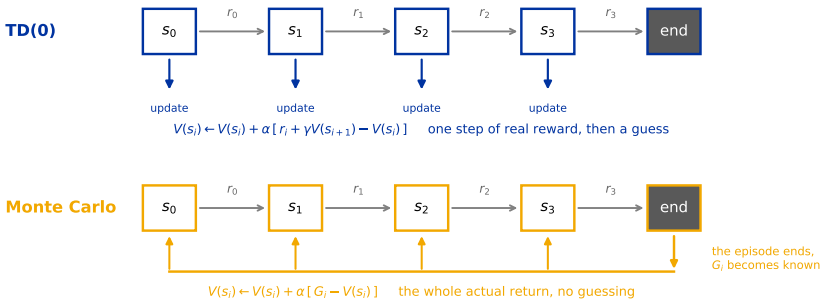
$$V(s_t) \leftarrow V(s_t) + \alpha\big[\underbrace{r_t + \gamma V(s_{t+1})}_{\text{TD target}} - V(s_t)\big]$$

the bracket is the **TD error** δ_t

Notice what just happened. We updated a guess towards another guess. That should feel illegitimate – and it is the single most useful idea in the subject. The one real number in the target, r_t, is enough to slowly drag the whole system towards the truth.

The same episode, two schedules

Same episode. The only question is when you are allowed to learn.



TD learns **online**, from incomplete episodes, in continuing tasks, and from every single step. Monte Carlo learns once, at the end, from the real thing.

The trade is bias against variance

	Monte Carlo	**TD(0)**
Target	G_t, the actual return	$r_t + \gamma V(s_{t+1})$
Bias	none	biased, because $V(s_{t+1})$ is wrong at first
Variance	high – sums every future step	low – one step of randomness
Needs	episodes that terminate	nothing; works online forever
Markov	does not assume it	exploits it (and suffers if it is false)

You have met this axis before – it is the bias-variance trade-off from chapter 2, in a new costume. And as usual the extremes are rarely best: in practice TD converges faster, and the bias washes out as the estimates improve.

Everything in between: n-step and TD(λ)

MC and TD(0) are the two ends of one dial. Look one step ahead, or two, or all of them:

Target	Name	
$r_t + \gamma V(s_{t+1})$	TD(0)	maximum bias, minimum variance
$r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$	2-step	
$r_t + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n})$	n-step	
$r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots$	Monte Carlo	minimum bias, maximum variance

TD(λ)

Rather than pick one n, take a weighted average of *all* of them, with weight $(1-\lambda)\lambda^{n-1}$. $\lambda = 0$ gives TD(0); $\lambda = 1$ gives Monte Carlo. **Eligibility traces** make this computable in one pass instead of storing the future.

Remember this dial. It comes back in lecture 4 as **GAE**, which is exactly this idea applied to advantages, and is a knob you will actually tune when training PPO.

Control, not just prediction

Estimating V^{π} is useless if you cannot act on it.

Why control forces you to learn Q

In the last lecture, going from values to a policy was one line:

$$\pi(s) = \arg\max_a \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a)\, V(s') \right]$$

There is P again. Without it, knowing V does **not** tell you how to act – you cannot work out where an action leads.

So learn $Q(s,a)$ instead

Q already has the action baked in, so acting greedily is just $\pi(s) = \arg\max_a Q(s,a)$ – a comparison between numbers you have already stored. No model required.

This is why every model-free algorithm in this chapter estimates Q or a policy directly, and almost never V alone.

The cost: the table is now $|\mathcal{S}| \times |\mathcal{A}|$ instead of $|\mathcal{S}|$. Hold that thought until lecture 3.

Generalised policy iteration, with samples

The skeleton from last lecture survives intact. Only the two boxes change:

	With a model (lecture 1)	**From samples (now)**
Evaluation	sweep the Bellman equation	average returns (MC) or bootstrap (TD)
Improvement	$\arg\max_a$ using P	$\arg\max_a Q(s, a)$, no P needed
Coverage	every state, every sweep	**only states you actually visit**

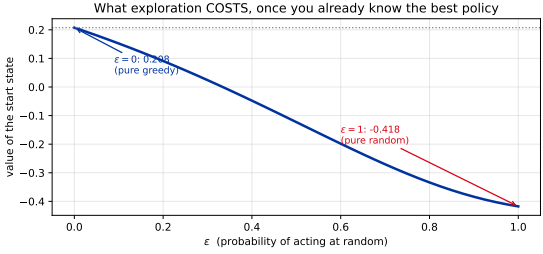
The new problem, and it is a serious one

Dynamic programming visited every state because it was allowed to. A sampling agent only learns about states it *reaches*, and it reaches them by following its own policy.

A greedy policy will therefore keep confirming its first impression and never look anywhere else. **Improvement now depends on exploration**, which was never true before.

The dilemma nobody escapes

To learn which action is best you must try actions that might not be. To score well you must take the action you already believe is best. You cannot do both at once.



ϵ-**greedy:** act randomly with probability ϵ, greedily otherwise.

Priced exactly on our gridworld, with the optimal policy already known:

- $\epsilon = 0$: value 0.208
- $\epsilon = 0.1$: value 0.152
- $\epsilon = 1$: value -0.418

Even 10% random acting costs about a quarter of the achievable value.

But that is only one side. The figure assumes you *already know* the best policy. At the start you do not, and $\epsilon = 0$ means never finding it. Hence the usual compromise: start high, decay over time.

Two updates that differ by one symbol

SARSA – on-policy

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[\, r + \gamma\, Q(s', a') - Q(s,a) \,\big]$$

a' is **the action you actually took next**. The name is literally the tuple it needs: **s**tate, **a**ction, **r**eward, **s**tate, **a**ction.

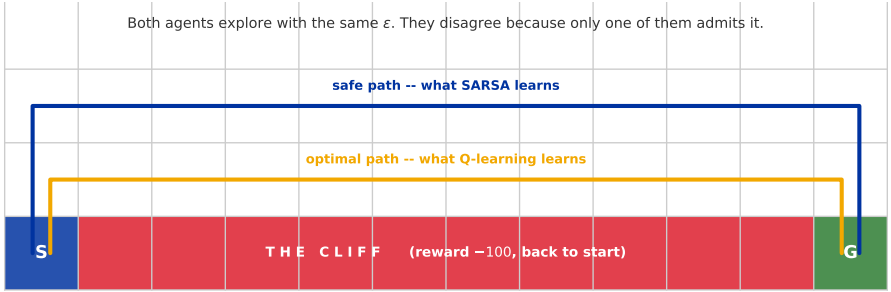
Q-learning – off-policy

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[\, r + \gamma \max_{a'} Q(s', a') - Q(s,a) \,\big]$$

The *best* action available there, whether or not you take it.

One symbol. It changes what the algorithm is learning about.

Why that one symbol matters



Q-learning learns the value of the greedy policy, which never falls off. So it walks the edge – and, while it is still exploring with ϵ, falls in regularly. *It scores worse online while learning the better policy.*

SARSA bootstraps off the action it will actually take, which is sometimes random. Squares next to the cliff therefore inherit some of the -100. It learns the best policy *for an agent that explores*.

If exploration can do real damage – a physical robot, a live recommender, a production system – the on-policy answer is usually the one you want, because it prices in the fact that your agent *will* sometimes do something random.

Expected SARSA, and the family so far

SARSA's target uses one sampled a', which adds noise for no good reason – you know $\pi(\cdot|s')$, so you can average over it exactly:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\, r + \gamma \sum_{a'} \pi(a'|s')\, Q(s',a') - Q(s,a) \,\Big]$$

Algorithm	**Target uses**	**Learns the value of**
SARSA	the sampled a'	the policy it is running
Expected SARSA	the average over $\pi(\cdot\|s')$	the policy it is running, less noisily
Q-learning	$\max_{a'}$	the greedy policy

Expected SARSA with a greedy target policy *is* Q-learning. These are not three algorithms; they are three choices of what distribution to average the target over. That framing is what makes the next section necessary.

Importance sampling

The most reused equation in this chapter.
Proximal Policy Optimization (**PPO**) is built on it.

The problem

You want to know how good policy π is. But your data was collected by a *different* policy b – and this situation is the rule, not the exception:

- You explore with ϵ-greedy but want the value of the greedy policy.
- You have last month's logs from the recommender that was deployed then.
- You want to reuse old experience instead of throwing it away after one gradient step.
- You collected a batch of responses from the model, then updated it, and want a second update from the same batch. *(This one is PPO.)*

Naively averaging b's returns estimates V^b, not V^π.
The samples are drawn from the wrong distribution.

Importance sampling, derived

The fix is one line of algebra: multiply and divide by q.

$$\begin{aligned}
\mathbb{E}_{x\sim p}[f(x)] &= \sum_x p(x) f(x) && \text{definition} \\
&= \sum_x q(x)\,\frac{p(x)}{q(x)}\,f(x) && \text{multiply by } q/q \\
&= \mathbb{E}_{x\sim q}\left[\frac{p(x)}{q(x)}\,f(x)\right] && \textbf{now sampled from } q
\end{aligned}$$

Applied to whole trajectories, something remarkable cancels:

$$\rho = \frac{\Pr(\tau \mid \pi)}{\Pr(\tau \mid b)} = \frac{\cancel{p(s_0)}\prod_t \pi(a_t|s_t)\,\cancel{P(s_{t+1}|s_t,a_t)}}{\cancel{p(s_0)}\prod_t b(a_t|s_t)\,\cancel{P(s_{t+1}|s_t,a_t)}} = \prod_t \frac{\pi(a_t|s_t)}{b(a_t|s_t)}$$

The dynamics cancel. The correction needs only the two policies – which you know exactly, because you wrote them.

The catch, and where you will see it again

That product is a time bomb

ρ multiplies T ratios together. If each is a little above 1, the product explodes; if a few are near 0, it collapses. Over a long trajectory the variance of the estimate becomes astronomical, and a single sample can dominate the entire batch.

Worse, ρ is undefined if b never takes an action π would – you cannot correct for data you do not have. (This requirement is called *coverage*.)

Every practical off-policy method is a way of managing that ratio:

Method	**What it does to ρ**
Q-learning	avoids it entirely – bootstraps off $\max_{a'} Q$, one step, no product
Lecture 4's policy methods	keep one *single-step* ratio, and **clip** it
Lecture 7's language-model methods	the same ratio, per token, with a group-based baseline

When you meet $\frac{\pi_\theta(a|s)}{\pi_{\text{old}}(a|s)}$ in the PPO objective, it is **this**, with the product cut down to one term.

Put it in the gridworld

Same world as lecture 1 – but this time nobody hands over P.

Q-learning: learn the values from experience alone

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[\underbrace{r + \gamma \max_{a'} Q(s',a')}_{\text{what I now think it is worth}} - \underbrace{Q(s,a)}_{\text{what I thought before}}\Big]$$

The bracket is a **surprise**: the gap between the estimate you had and the better estimate you can make now that you have seen r and s'. Nudge towards it by α.

Model-free
Never needs P. Just (s, a, r, s').

Off-policy
Learns Q^* even while behaving badly.

Tabular
One cell per (s, a). Does not scale.

Watkins & Dayan (1992). Converges to Q^* given decaying α and infinitely many visits to every pair – conditions no real system satisfies, and it usually works anyway.

One update, by hand

Suppose $\alpha = 0.5$, $\gamma = 0.9$. The agent is on a square where it currently believes $Q(s, \text{right}) = 0.30$. It moves right, receives the usual -0.04, and lands on a square whose best action it currently values at 0.60.

$$\text{target} \;=\; r + \gamma \max_{a'} Q(s', a') \;=\; -0.04 + 0.9 \times 0.60 \;=\; \mathbf{0.50}$$

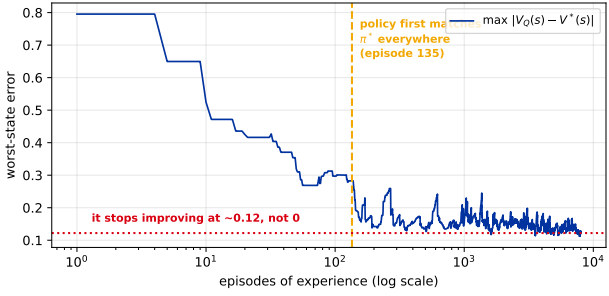
$$\text{surprise} \;=\; 0.50 - 0.30 \;=\; +0.20$$

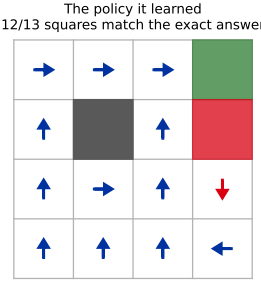
$$Q(s, \text{right}) \;\leftarrow\; 0.30 + 0.5 \times 0.20 \;=\; \mathbf{0.40}$$

That is the whole algorithm. Everything else in tabular RL is bookkeeping around this line – and notice it never once asked where the agent was likely to end up. It only used where it *did* end up.

Does it actually work?

Lecture 1 was *planning*: value iteration used $P(s'|s,a)$. So put an agent in the same gridworld, give it **only** the ability to take a step and see what happens, and let it run.





From zero knowledge, its policy matches the exact answer everywhere by episode **135**, and ends up right in **12 of 13** squares. The one it misses (red) is a near-tie: the two actions differ in true value by 0.018, and – the part that matters – it still does **not** walk past the pit.

But look where the error stops

The curve flattens at about 0.12 and stays there. More experience does not fix it. Why would a method that provably converges get stuck above zero?

Because the error is not random. Measured across all 13 squares at the end of training:

13 **of** 13 **states are overestimated.** Mean $+0.079$, worst $+0.128$.

Maximisation bias

Look at the update rule again: the target contains $\max_{a'} Q(s', a')$. Taking a *maximum* over estimates that each carry noise does not give the maximum of the true values – it gives something systematically too high, because the max picks whichever action happened to get lucky. Every state then inherits the optimism of the states after it, so the bias compounds backwards.

The standard fix, **Double Q-learning** (van Hasselt, 2010), keeps two independent estimates and uses one to *choose* the action and the other to *value* it – so the noise cannot select and reward itself. It returns in lecture 3 as Double DQN.

When is it allowed to work?

Q-learning converges to Q^* under three conditions. They are worth knowing because every one of them is violated in practice:

Condition	Why	Reality
Every (s, a) visited infinitely often	you cannot learn about what you never try	you visit a vanishing fraction of a real state space
$\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$	big enough to reach anywhere, small enough to settle	people use a constant α, which fails the second
Tabular representation	each cell updates independently	lecture 3 replaces the table with a network

This is not pedantry. A first attempt at the gridworld demo with constant $\alpha = 0.2$ matched π^* at episode 491 and then *drifted back* to 6 of 13 squares wrong. The decaying schedule is what fixed it.

Recap

- Without P, every expectation becomes an average over samples. That single substitution is what separates this lecture from the last one.
- **Monte Carlo** averages actual returns: unbiased, high variance, needs the episode to end. **TD** bootstraps off its own estimate: biased, low variance, learns online.
- n-step and TD(λ) are the dial between them. The same dial reappears as **GAE** in lecture 4.
- Control forces you to learn **Q**, because acting from V needs the model you do not have.
- **SARSA** learns the value of the policy it is running; **Q-learning** learns the value of the greedy one. On the cliff, that is the difference between the safe path and the edge.
- **Importance sampling** corrects for data collected by another policy, and the dynamics cancel out of the ratio. **Proximal Policy Optimization – and every language-model method in lectures 6 and 7 – is built on the one-step version of it.**
- Tabular Q-learning really does recover π^* from experience alone – and it overestimates every state, because max over noisy estimates is biased upward.

Next: the table has one cell per state. Atari has more states than the universe has atoms. Function approximation, and the three ingredients that make it diverge.